# NHAI Offline Facial Recognition and Liveness Detection System

### NHAI Innovation Hackathon 7.0 Submission Proposal

**Repository:** https://github.com/bhavikdeshmukh/NHAI-Offline-Facial-Recognition-System
**Objective:** Offline, lightweight, mobile-ready face verification and liveness detection for Datalake 3.0 field operations in zero-network zones.

## Challenge Alignment

This submission addresses the official objective: a highly accurate, lightweight, and entirely offline facial recognition and liveness detection system that can integrate into the existing Datalake 3.0 app. The prototype demonstrates the complete field workflow, while the documentation maps the browser implementation to a React Native production architecture.

## Current Prototype Deliverables

- Camera-based officer enrollment with center, left, and right samples.
- Blur and lighting quality gates before saving a local template.
- Offline verification with immediate App Unlocked or Face ID Not Matched result.
- Local storage simulating on-device template storage.
- Pending audit queue and Vercel /api/sync simulation for Datalake 3.0 recovery.
- CLAHE lighting evidence, real-image face crop evidence, benchmark CSV files, and technical documentation.

## Production Architecture

```
React Native Camera
```

```
-> Frame Processor

-> CLAHE Lighting Normalization

-> Face Detection and 112x112 Face Crop

-> Passive Liveness Check

-> Active Liveness Challenge

-> MobileFaceNet INT8 Embedding

-> Cosine Similarity Decision

-> Encrypted SQLite Template Store

-> Offline Audit Queue

-> Datalake 3.0 / AWS Sync and Purge
```

## Model Strategy

| Component | Planned Model or Method | Purpose |
|---|---|---|
| Face detection | BlazeFace or SCRFD-lite | Fast mobile face localization |
| Preprocessing | CLAHE | Robustness under sunlight, shadows, low light, and blur |
| Embedding | MobileFaceNet INT8 | Lightweight face vector generation under model-size budget |

| | | |
|---|---|---|
| Matching | Cosine similarity | Fully offline template comparison |
| Passive liveness | FASNet/Silent-Face style model | Detect photo, print, and screen replay attacks |
| Active liveness | Blink, head turn, or smile prompt | Confirm live human participation |
| Explainability | Grad-CAM style evidence | Improve reviewer trust in model decisions |

## Evaluation Criteria Mapping

### Innovation Level: 30 Marks

- Edge AI design instead of server-dependent authentication.
- MobileFaceNet INT8 target under the approximate 20 MB model footprint.
- CLAHE preprocessing for Indian outdoor field conditions.
- Passive plus active liveness design, stronger than blink-only liveness.
- Real-image preprocessing and face-crop evidence included in the repository.

### Feasibility: 30 Marks

- Prototype runs immediately in browser and works on HTTPS-enabled phone browsers.
- Workflow maps cleanly to React Native camera, model, storage, and sync modules.
- Offline cosine matching and local template storage are already demonstrated.
- Target runtime is under 1 second on mid-range devices.

### Scalability and Sustainability: 20 Marks

- Offline queue stores audit events when network is unavailable.
- Sync simulation shows records being pushed after connectivity returns.
- Production design includes purge after successful sync.
- Multi-sample enrollment and CLAHE improve robustness across pose and lighting variation.

**Presentation and Documentation: 20 Marks**

- Clean README, setup guide, architecture guide, model card, benchmark document, and final evaluation map.
- Benchmark CSV files and evidence assets avoid unsupported claims.
- Demo video shows the user journey from enrollment to unlock and sync.

## Honest Prototype Boundary

The current web demo uses a deterministic temporary embedding to prove the offline enrollment, vector comparison, lock/unlock decision, and sync workflow. It does not claim final MobileFaceNet recognition accuracy. The final recognition layer is designed to replace this temporary embedding with a quantized MobileFaceNet TFLite model in the React Native implementation.

## Recommended Demo Flow

1. Show the GitHub README and architecture.
2. Open the hosted demo link.
3. Enroll with center, left, and right samples.
4. Return to verification and show the locked state.
5. Tap Analyze and Unlock.
6. Show App Unlocked and match latency under one second.
7. Open Sync Details and simulate online sync.

## Conclusion

The submission presents a practical offline-first identity verification system for NHAI field conditions. It emphasizes lightweight edge AI, liveness protection, local auditability, robust lighting handling, and a clear integration path into Datalake 3.0.